JAVA INTERVIEW PREPARATION

CHAPTER 17

HASHMAP INTERNALS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

HashMap Internals: Hashing, Collisions, Resizing, and Key Correctness

Senior / Lead Java Developer Reading Chapter

HashMap is often introduced as a collection that provides constant-time lookup. That description is useful but incomplete. A senior developer should understand what the constant-time claim assumes, how keys are converted into bucket locations, what happens when several keys share a bucket, why resizing can create latency and allocation pressure, and how a broken key class can make entries appear to disappear.

HashMap is an array-based hash table. The array contains buckets, and each bucket is empty or points to one or more entries. An entry stores a hash, key, value, and link or tree relationship to other entries in the same bucket.

```
table array
 index 0  -> empty
 index 1  -> [hash, key A, value] -> [hash, key B, value]
 index 2  -> [hash, key C, value]
 index 3  -> empty
```

The map does not search every key. It narrows the search to one bucket and then uses equality within that bucket.

From Key to Bucket

A lookup conceptually performs four operations:

1. Obtain the key's hashCode. 2. Spread information from high hash bits into lower bits. 3. Convert the spread hash to an array index. 4. Search that bucket using hash comparison and equals.

When the table length is a power of two, the index can be calculated efficiently:

```
index = (tableLength - 1) & spreadHash
```

The bit mask is faster than a general modulo and distributes useful low bits across the available indexes. HashMap uses power-of-two capacities so this rule remains valid after resizing.

The spreading step matters because some application hash functions vary mainly in their high bits. Without spreading, a small table would ignore those bits and produce avoidable collisions. Spreading improves imperfect hashes; it cannot repair a hashCode that always returns the same value.

How Lookup Confirms a Key

The bucket index only identifies a candidate group. Different keys can legally have the same hash. HashMap first checks the stored hash and then key identity or equality.

```
if (storedHash == requestedHash &&
        (storedKey == requestedKey || requestedKey.equals(storedKey))) {
    return storedValue;
}
```

This is why equal objects must have equal hash codes. If two equal keys produce different hashes, lookup may inspect a different bucket and never call equals on the stored key.

Unequal objects may share a hash. That is a collision, not a contract violation. Performance depends on collisions being reasonably distributed rather than absent.

Insertion and Replacement

Putting an entry follows the same bucket selection. If an equal key already exists, HashMap replaces its value and returns the old value. The key object already stored in the map normally remains the bucket's key.

```
Map<CustomerId, Customer> customers = new HashMap<>();
customers.put(new CustomerId("C-17"), oldCustomer);
customers.put(new CustomerId("C-17"), revisedCustomer);

// One logical entry, provided CustomerId implements equality correctly.
```

If no equal key exists, a new entry is added. Size increases, and the map may resize when size passes its threshold.

Collision Chains and Tree Bins

Traditionally, colliding entries form a linked list. A long list makes lookup linear in the number of entries in that bucket. Modern HashMap can convert a heavily populated bucket into a balanced tree, improving severe-collision lookup toward logarithmic behavior.

```
normal bucket                 treeified bucket
 A -> B -> C                        D
                                   / \
                                  B   F
                                 / \ / \
                                A  C E  G
```

In current OpenJDK implementations, treeification is associated with a bucket reaching eight entries, but only when the table has reached a minimum capacity of 64. For a smaller table, resizing is preferred because more buckets may distribute the entries naturally. A tree can be converted back when the bucket becomes small; six is the implementation's untreeification threshold.

These numbers are implementation details rather than promises of the Map interface. They are valuable for explaining behavior, but application correctness must not depend on them.

Capacity, Load Factor, and Threshold

Capacity is the bucket-array length. Load factor controls how full the table is allowed to become before growth. The common default load factor is 0.75, balancing memory against collisions.

```
threshold approximately = capacity x load factor

capacity 16, load factor 0.75 -> resize near size 12
```

A lower load factor uses more memory but tends to reduce collisions. A higher factor saves bucket-array space but increases bucket density. Changing it is rarely the first performance optimization; key quality and realistic measurement matter more.

When the expected entry count is known, initial sizing avoids repeated resize work. Do not simply pass the expected size as capacity when using a 0.75 load factor. Allow room for the threshold and power-of-two rounding.

```
int expectedEntries = 100_000;
Map<String, Order> orders = HashMap.newHashMap(expectedEntries); // modern JDK
```

Where the target JDK does not provide that factory, calculate or use a safely oversized initial capacity without overflowing.

What Resizing Actually Costs

When the threshold is exceeded, HashMap allocates a larger table, normally doubling capacity, and redistributes existing entries. Because the capacity doubles, an entry either stays at its old index or moves by the old capacity, based on one additional hash bit.

```
old capacity 16
old bucket 5 entries split into:
  new bucket 5
  new bucket 21  (5 + 16)
```

The spread hashes do not need to be recomputed from the keys, but every existing entry must still participate in transfer. A large resize creates an allocation and latency spike. Repeated growth during request processing can increase garbage collection and tail latency.

Mutable Keys: The Disappearing Entry

The most dangerous HashMap bug is changing fields used by equals or hashCode while the object is a key.

```
final class AccountKey {
    String accountNumber;

    // equals and hashCode use accountNumber
}

AccountKey key = new AccountKey();
key.accountNumber = "A-1";
map.put(key, "active");

key.accountNumber = "A-2";
map.get(key); // may return null
```

The entry is physically stored in the bucket selected by the old hash. Lookup uses the new hash and searches another bucket. Even iteration may reveal the entry while get cannot find it. Keys should be immutable value objects, records whose components are themselves stable, enums, strings, or other types with stable equality state.

Nulls and Ambiguous Lookup

HashMap permits one null key and multiple null values. A null key is handled specially and maps to a defined bucket path. Because a present key may map to null, get alone cannot distinguish absence.

```
if (map.get(key) == null) {
    // key may be absent, or present with a null value
}

boolean present = map.containsKey(key);
```

Where possible, avoid null values and make absence explicit in the surrounding API. Do not put Optional values in a map by default; often the map's absence semantics already express the missing case.

Iteration and Fail-Fast Behavior

HashMap iteration walks the bucket array and its entries. It does not promise insertion order or sorted order. Iteration cost is related to capacity plus size, so an enormously oversized sparse map can be expensive to traverse.

Iterators are fail-fast on a best-effort basis. A structural modification outside the iterator may cause ConcurrentModificationException. This is a bug detector, not a thread-safety guarantee.

```
for (String key : map.keySet()) {
    if (shouldRemove(key)) {
        map.remove(key); // unsafe during this iteration
    }
}
```

Use Iterator.remove, removeIf, or a separate collection of changes. Concurrent access requires a suitable concurrency design, not reliance on fail-fast behavior.

HashMap Is Not Thread-Safe

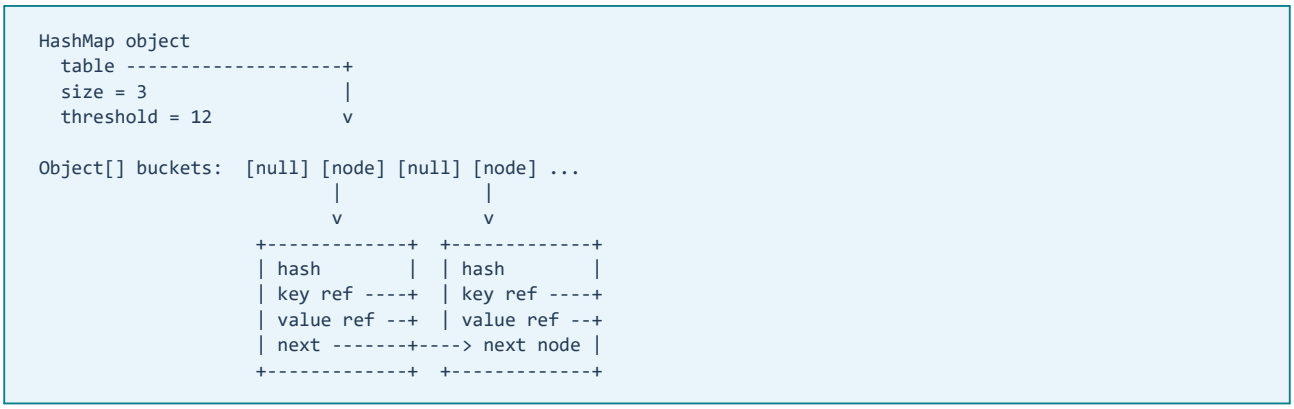
Concurrent reads are only safe when the map is safely published and no thread mutates it afterward. Concurrent modification without synchronization can cause visibility problems, lost updates, inconsistent observations, and invalid compound operations.

```
if (!map.containsKey(key)) {
    map.put(key, createValue());
}
```

Even if individual methods were synchronized separately, this check-then-act sequence would not be atomic. Use ConcurrentHashMap atomic APIs, an external lock covering the whole invariant, or construct an immutable map before publication.

The Entry Object and Its Memory Cost

The table does not store keys and values directly in adjacent array slots. Each occupied mapping is normally represented by a node object containing the spread hash, references to the key and value, and a link to the next node. The bucket array stores references to the first node in each bin.



Memory therefore includes the map object, bucket array, one node per mapping, keys, values, and any collision-tree nodes. Boxing primitive identifiers adds more objects. For millions of small mappings, structural overhead may exceed the business data. Heap analysis should distinguish retained key/value payload from table and node overhead.

`clear()` removes references from the table so entries can be reclaimed, but the current table capacity is generally retained. Clearing a once-huge map does not necessarily return its large bucket array. If the map will remain small, replacement with a new appropriately sized instance may release that capacity after safe publication and garbage collection.

Why the Hash Is Spread

In the Java 21 implementation, the spread operation is conceptually:

```
int h = key.hashCode();
int spread = h ^ (h >>> 16);
int index = (table.length - 1) & spread;
```

The right shift is unsigned. XOR folds information from the upper half of the 32-bit hash into the lower half, which matters because a small power-of-two table uses only a limited number of low bits for indexing.

```
original hash: [ high 16 bits ][ low 16 bits ]
shifted:       [      zeros    ][ high 16 bits]
XOR:           [ high 16 bits ][ mixed low bits]
                                    |
                              bucket mask uses these
```

The stored spread hash also avoids calling a potentially expensive or unstable `hashCode()` repeatedly while traversing a bin. This is an implementation explanation, not permission for mutable keys: lookup still recomputes the requested key's hash to find the bin.

Resizing Uses One Additional Bit

When a power-of-two capacity doubles from `oldCap` to `2 * oldCap`, existing nodes in an old bin do not move arbitrarily. The old mask already determined the low bits. Only the bit represented by `oldCap` decides whether an entry remains at index `i` or moves to `i + oldCap`.

```
old capacity = 8       old mask = 0000 0111
new capacity = 16      new mask = 0000 1111

spread hash ...0xxx  -> old bucket i, stays at i
spread hash ...1xxx  -> old bucket i, moves to i + 8
                ^
          newly observed bit
```

This split is cheaper than recomputing a general modulo, but transfer is still proportional to the number of existing mappings plus bucket traversal. A request that crosses a large threshold pays an unusual one-time cost, which can appear as a latency outlier even though average insertion remains expected constant time.

Pre-sizing should reflect the expected number of mappings, not blindly allocate the largest conceivable map. Oversizing increases memory and iteration cost. In Java 21, `HashMap.newHashMap(expectedMappings)` expresses expected mappings directly and accounts for implementation sizing rules. When maintaining code for older JDKs, document any manual capacity calculation and protect it from integer overflow.

Tree Bins Are a Defensive Optimization

Tree bins use red-black-tree mechanics while preserving a linked traversal relationship. Hash is still the primary discriminator. When hashes collide, comparable keys of a compatible class can help order nodes; the implementation also has tie-breaking logic when natural comparison cannot provide an order.

Do not conclude that keys must implement `Comparable` for HashMap correctness. `equals` and `hashCode` define map identity. Comparability may help tree-bin navigation under collisions, but it must be consistent and must not be introduced merely to compensate for a poor hash function.

```
ordinary case:       good distribution -> short bins -> expected O(1)
collision defense:   long bin + table >= 64 -> tree -> toward O(log n)
design goal:         tree bins should be rare, not the normal state
```

Treeification costs memory because tree nodes carry more links and balancing metadata. A profile showing many tree bins suggests hostile input or systematic hash-quality problems worthy of investigation.

Backed Views Are Live, Not Snapshots

`keySet()`, `values()`, and `entrySet()` return views backed by the map. Removing through a supported view removes the mapping. Later map changes are reflected in the view.

```
Map<String, Integer> counts = new HashMap<>();
counts.put("open", 4);

Set<String> keys = counts.keySet();
keys.remove("open");       // also removes the map entry
```

The key view cannot add a key because there is no value to associate with it. An entry returned during iteration can expose `setValue`, subject to the view and iterator contract. If a stable snapshot is required, copy explicitly and decide whether the keys and values themselves also require defensive copies.

Compute and Merge Methods Encode State Transitions

Methods such as `computeIfAbsent`, `compute`, and `merge` make single-map transformations clearer, but HashMap remains non-concurrent. They do not make unsynchronized access thread-safe.

```
wordCounts.merge(word, 1, Integer::sum);

groups.computeIfAbsent(team, ignored -> new ArrayList<>())
      .add(employee);
```

For HashMap, a null result from a remapping function can mean “do not record” or “remove the mapping,” depending on the method. Mapping functions should not recursively modify the same map; the implementation may detect some interference and throw `ConcurrentModificationException`, but correctness must not depend on detection.

The multivalue example also exposes an ownership issue: the list stored as a value remains mutable. Returning the map directly can allow callers to mutate internal groups without using map APIs. Collection choice and encapsulation still matter after the convenient initialization step.

Choosing the Right Map

HashMap is appropriate when exact lookup dominates and ordering is irrelevant. Select a different structure when the invariant demands more:

```
requirement                        likely choice
-----------------------------------------------------------
insertion or access order          LinkedHashMap
sorted keys and range operations   TreeMap
enum keys                          EnumMap
concurrent updates                 ConcurrentHashMap
identity rather than equals        IdentityHashMap (specialized)
immutable published configuration  Map.copyOf / Map.of entries
```

Choosing HashMap and later depending on its observed iteration order is a common latent defect. A particular order can appear stable for months and change after a resize, key-set change, or JDK implementation change because no ordering contract exists.

A Strong Interview Explanation

Start with the complete lookup path: HashMap spreads `hashCode`, masks it into a power-of-two bucket array, then confirms a candidate using stored hash and key equality. Explain that expected constant time assumes good distribution; collisions are valid and are handled first by linked bins and, in modern implementations, by tree bins under implementation-specific conditions.

Then connect the mechanics to engineering decisions. Resizing doubles capacity and transfers entries, so pre-sizing can reduce tail-latency and allocation spikes. Keys must keep equality-relevant state stable. Iteration is proportional to capacity plus size, backed views are live, and fail-fast behavior is not synchronization. Finish by saying that map choice follows ordering, concurrency, memory, and range-query requirements rather than habit.

Performance and Security Perspective

Expected get and put cost is constant under a well-distributed hash. Worst-case behavior is influenced by collision structure, key comparability, table capacity, and implementation rules. Tree bins reduce the impact of deliberate or accidental collision floods, but they do not make poor keys free.

For production diagnosis, inspect allocation profiles, resize frequency, map size, key retention, hash distribution, and equality cost. A hashCode that walks a large object graph can dominate lookup time. A cache backed by an unbounded HashMap is also a memory leak by policy: the map works correctly while retaining everything forever.

Interview-Ready Summary

HashMap is a power-of-two bucket array. It spreads a key's hash, masks it to a bucket index, and resolves collisions by comparing stored hashes and keys. Equal keys must have equal hash codes; collisions between unequal keys are valid.

Buckets begin as lists and may become balanced trees under heavy collision when the table is sufficiently large. The default load factor is commonly 0.75. Resizing doubles the table and redistributes entries, so correct initial sizing can reduce allocation and tail-latency spikes.

Keys must not change equality-relevant state while stored. HashMap permits nulls, gives no ordering guarantee, and is not thread-safe. Fail-fast iteration is diagnostic rather than a concurrency mechanism.

Revision Notes

- HashMap spreads a hash into a power-of-two bucket index, then confirms candidates with equality.
- Equal objects require equal hash codes; unequal objects may legally collide.
- A put for an equal key replaces the existing value.
- The default load factor is 0.75; resize normally doubles capacity and transfers entries.
- Large resizes can affect allocation, GC, and tail latency.
- Long collision lists may treeify, but thresholds are implementation details.
- Immutable keys prevent entries from becoming unreachable by lookup.
- HashMap permits nulls, so `get` returning null does not prove absence.
- Iteration order is unspecified and iteration cost includes table capacity.
- Fail-fast iterators are best-effort bug detection.
- HashMap is unsafe for concurrent mutation; immutable-after-publication use requires safe publication.
- Measure hash distribution, equality cost, resizing, and retained map size in production.
- Each mapping normally adds a node; `clear()` generally retains bucket capacity.
- Hash spreading folds high bits down before masking.
- On resize, one new hash bit decides whether a node stays or moves by old capacity.
- Tree bins are collision defense and should be rare with good keys.
- `keySet`, `values`, and `entrySet` are live backed views, not snapshots.
- Compute and merge methods do not make HashMap safe for concurrent access.
- Select LinkedHashMap, TreeMap, EnumMap, or ConcurrentHashMap when their contract fits better.